

Phase 6: Project Testing

Model Evaluation

As part of train.py, three classification models were trained on an 80/20 stratified train-test split of the loan approval dataset and evaluated on the held-out 20% test set (854 records). The comparison below shows the actual results obtained; Random Forest achieved the highest accuracy and was automatically selected as the deployed model (saved as loan_model.joblib).

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	93.56%	90.61%	92.57%	91.58%
Random Forest (Selected)	98.13%	99.04%	95.98%	97.48%
XGBoost	98.01%	98.42%	96.28%	97.34%

Random Forest was selected as the final model because it achieved the highest overall accuracy (98.13%) and the best balance of precision and recall among the three candidates, indicating that it correctly identifies both approved and rejected applications with very few misclassifications.

System Testing

In addition to model-level evaluation, the end-to-end web application was tested by actually running app.py locally and sending real requests to the /predict API endpoint (via curl), to confirm the system behaves correctly for normal predictions, boundary cases, and invalid input. The table below shows the exact inputs used and the actual responses returned by the running application (not simulated or assumed):

#	Test Case	Sample Input	HTTP Status	Actual Result
1	High CIBIL score, sufficient income and assets	CIBIL: 780, Income: ₹90L, low dependents	200 OK	Approved (99.0%)
2	Low CIBIL score with high loan amount requested	CIBIL: 450, Loan Amount: ₹35L	200 OK	Rejected (93.0%)
3	Missing required field (bank_assets omitted)	One required field left out of the request	400 Error	Error returned, no crash
4	Boundary CIBIL score around 600	CIBIL: 600, moderate	200 OK	Approved

#	Test Case	Sample Input	HTTP Status	Actual Result
		income/assets		(92.0%)
5	Valid input with extreme high asset values	Assets in the ₹4–12 crore range	200 OK	Approved (90.0%)

All five test cases behaved as expected: valid inputs returned a 200 OK response with a correct Approved/Rejected classification and probability, and the missing-field case correctly returned an error response instead of crashing the server, confirming that the API's error handling works as intended.

Developer: Ch. Srilatha | [GitHub: @Srilatha-dot](#)